



Universidad Nacional Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Unidad Curricular: Lenguaje Y Compiladores

TEMA 5:

Análisis Teórico Para AST, LL/LR, Generadores Sintácticos Y Manejo De Errores

Elaborado por:

Profesor(a):

Félix Márquez

Shirley Cedeño V-30.413.183

Rayc Yanez V- 28.215.618

Ángel Rodríguez V-30.437.205

Nelson Bueno V-27.407.263

Ciudad Guayana 26/07/2026

Introducción

El diseño de analizadores sintácticos y la estructuración de representaciones intermedias constituyen el núcleo operativo de la teoría de compiladores, actuando como la frontera lógica entre el código fuente y el procesamiento de la máquina. Tradicionalmente, la arquitectura de estos sistemas se ha centrado en la rigurosidad formal, empleando algoritmos deterministas capaces de validar estructuras gramaticales y construir derivaciones exactas de manera eficiente. Sin embargo, el estudio teórico de estos componentes demuestra que la robustez de un compilador no solo depende de su tolerancia en el procesamiento de sintaxis válida, sino en su capacidad arquitectónica para abstraer la semántica real y gestionar las discrepancias de manera resiliente.

Ante este panorama, el presente informe documenta la evaluación teórica de los paradigmas de análisis sintáctico desde una perspectiva estructural y comparativa. En primer lugar, se expone el Árbol de Sintaxis Abstracta (AST) como el mecanismo definitivo de representación jerárquica, capaz de destilar la topología lógica de un programa omitiendo la redundancia impuesta por la gramática. En segundo lugar, se plantea un análisis profundo entre las familias algorítmicas de análisis descendente (LL) y ascendente (LR), evaluando sus estrategias de derivación, complejidad matemática y su vigencia en la industria del software moderno. Finalmente, se aborda la evolución de los metacompiladores contemporáneos, analizando cómo los generadores modernos abstraen la complejidad de los autómatas y aplican estrategias sofisticadas para la mitigación y recuperación de errores sintácticos.

1. Árbol de Sintaxis Abstracta (AST)

1.1. Definición

Un Árbol de Sintaxis Abstracta (AST, por sus siglas en inglés) es una estructura de datos jerárquica generada por el analizador sintáctico (parser) que representa la topología y jerarquía lógica de un programa fuente. A diferencia del árbol de análisis sintáctico concreto (Parse Tree), el AST es una versión "destilada" que omite los tokens estructurales redundantes exigidos por la Gramática Libre de Contexto (GLC) —tales como paréntesis, llaves, comas o puntos y comas—, conservando exclusivamente las operaciones, operandos y sentencias que poseen un impacto directo en la semántica del programa.

1.2. Características

- **Abstracción de Formato:** Descarta los lexemas de puntuación y agrupamiento. La precedencia de operadores se codifica de forma natural mediante la profundidad de los nodos en el árbol.
- **Topología Jerárquica:** Se compone de un nodo raíz que se bifurca en nodos internos (los cuales representan operaciones, bifurcaciones de control o asignaciones) y nodos hoja (que representan los elementos atómicos, como literales numéricos o identificadores de variables).
- **Independencia de la Gramática:** Diferentes gramáticas formales (por ejemplo, una gramática LL no ambigua y una gramática LR con recursión izquierda) pueden ser programadas para producir un AST estructuralmente idéntico para el mismo bloque de código.
- **Base para el Backend:** Sirve como la representación intermedia definitiva que el compilador utilizará en sus fases subsecuentes (análisis semántico, optimización y generación de código de máquina).

1.3. Ejemplos de Entrada y Análisis de la Representación Matemática

El Árbol de Sintaxis Abstracta brilla especialmente en la evaluación de expresiones matemáticas. Su estructura garantiza que las reglas del álgebra (como la jerarquía de operaciones y la asociatividad) se cumplan de manera estricta sin necesidad de almacenar los caracteres de agrupación (paréntesis) utilizados en el código fuente. La precedencia matemática se traslada directamente a la topología geométrica del árbol: la profundidad de los nodos dictamina el orden de evaluación, resolviéndose primero los nodos más profundos.

Para ilustrar este principio, analicemos dos escenarios matemáticos distintos:

Ejemplo A: Expresión Matemática con Alteración de Precedencia (Uso de Paréntesis)

Entrada: $x = (a + b) * c;$

Análisis Matemático:

En las reglas del álgebra estándar, la multiplicación (*) tiene una precedencia superior a la suma (+). Sin embargo, los paréntesis obligan al sistema a evaluar primero el bloque (a + b). En el AST, esta alteración forzada se refleja colocando el nodo de la suma en un nivel de profundidad mayor que el nodo de la multiplicación. Al evaluar el árbol mediante un recorrido de abajo hacia arriba (recorrido en postorden), el motor resuelve primero la operación inferior.

- **Nivel 0 (Raíz):** Nodo Asignacion (=) -> Fija el resultado matemático en la variable de memoria.
 - **Nivel 1 - Hijo Izquierdo:** Nodo Identificador (x) -> Variable destino.
 - **Nivel 1 - Hijo Derecho:** Nodo OperacionBinaria (*) -> Es la operación principal que engloba toda la parte derecha de la ecuación.
 - **Nivel 2 - Hijo Izquierdo (del):* Nodo OperacionBinaria (+) -> La suma se coloca aquí abajo para que se evalúe antes de poder multiplicar.
 - **Nivel 3 - Hijo Izq (del +):** Nodo Identificador (a)
 - **Nivel 3 - Hijo Der (del +):** Nodo Identificador (b)
 - **Nivel 2 - Hijo Derecho (del):* Nodo Identificador (c)

Ejemplo B: Precedencia Natural Sin Agrupación

Entrada: $z = a + b * c;$

Análisis Matemático:

En este caso, la sintaxis no posee paréntesis delimitadores. El analizador sintáctico, guiado por las reglas matemáticas de la gramática, detecta que la multiplicación debe resolverse incondicionalmente antes que la suma. Por lo tanto, el árbol se invierte en comparación al Ejemplo A: el AST ubica la multiplicación en el nivel inferior (aislando b y c), y deja la suma en la cúpula superior para que, como último paso matemático, se agregue el valor de a al producto ya calculado.

- **Nivel 0 (Raíz):** Nodo Asignacion (=)
 - **Nivel 1 - Hijo Izq:** Nodo Identificador (z)
 - **Nivel 1 - Hijo Der:** Nodo OperacionBinaria (+) -> La suma sube de nivel, siendo la última operación en resolverse.
 - **Nivel 2 - Hijo Izq (del +):** Nodo Identificador (a)
 - **Nivel 2 - Hijo Der (del +):** Nodo OperacionBinaria (*) -> Se evalúa primero por su profundidad topológica.
 - **Nivel 3 - Hijo Izq (del):* Nodo Identificador (b)
 - **Nivel 3 - Hijo Der (del):* Nodo Identificador (c)

1.4. Implementación del Árbol de Sintaxis Abstracta (Python)

A continuación, se presenta la implementación de los nodos y la construcción manual del AST para el Ejemplo A ($x = (a + b) * c$;). Python permite estructurar esta jerarquía de forma muy limpia mediante clases.

```
class NodoAST:
```

```
    """Clase base abstracta para todos los nodos del AST."""
```

```
    def imprimir(self, indentacion=""):
```

```
        pass
```

```
class Identificador(NodoAST):
```

```
    """Nodo hoja para representar variables."""
```

```
    def __init__(self, nombre):
```

```
        self.nombre = nombre
```

```
    def imprimir(self, indentacion=""):
```

```
        print(f"{indentacion}Identificador({self.nombre})")
```

```
class OperacionBinaria(NodoAST):
```

```
    """Nodo interno para representar operaciones aritméticas."""
```

```
    def __init__(self, operador, izquierda, derecha):
```

```
        self.operador = operador
```

```
        self.izquierda = izquierda
```

```
        self.derecha = derecha
```

```
    def imprimir(self, indentacion=""):
```

```
print(f"{indentacion}OperacionBinaria('{self.operador}')
```

```
self.izquierda.imprimir(indentacion + " ")
```

```
self.derecha.imprimir(indentacion + " ")
```

```
class Asignacion(NodoAST):
```

```
    """Nodo interno para representar la asignación a una variable."""
```

```
    def __init__(self, variable, expresion):
```

```
        self.variable = variable
```

```
        self.expresion = expresion
```

```
    def imprimir(self, indentacion=""):
```

```
        print(f"{indentacion}Asignacion('=')
```

```
        self.variable.imprimir(indentacion + " ")
```

```
        self.expresion.imprimir(indentacion + " ")
```

```
# Ejecución del programa
```

```
if __name__ == "__main__":
```

```
    # Construcción del AST para la entrada:  $x = (a + b) * c$ 
```

```
    # 1. Instanciación de los nodos hoja (Variables)
```

```
    nodo_a = Identificador("a")
```

```
    nodo_b = Identificador("b")
```

```
    nodo_c = Identificador("c")
```

```
    nodo_x = Identificador("x")
```

2. Construcción de abajo hacia arriba respetando la precedencia matemática

```
nodo_suma = OperacionBinaria("+", nodo_a, nodo_b)
```

```
nodo_multiplicacion = OperacionBinaria("*", nodo_suma, nodo_c)
```

```
ast_raiz = Asignacion(nodo_x, nodo_multiplicacion)
```

3. Visualizar la estructura jerárquica

```
print("Estructura del Árbol de Sintaxis Abstracta:")
```

```
ast_raiz.imprimir()
```

2. Comparación entre los Análisis LL y LR

2.1. Comparativa Teórica

Los analizadores sintácticos se dividen fundamentalmente en dos familias topológicas según cómo procesan las reglas de la gramática y construyen el Árbol de Sintaxis Abstracta (AST): la aproximación descendente (LL) y la ascendente (LR).

Criterio de Evaluación	Análisis LL (Descendente)	Análisis LR (Ascendente)
Estrategia de Construcción	Construye el árbol desde la raíz hacia las hojas (<i>Top-down</i>).	Construye el árbol desde las hojas hacia la raíz (<i>Bottom-up</i>).
Lectura y Derivación	Left-to-right, Leftmost derivation (Derivación por la izquierda).	Left-to-right, Rightmost derivation (Derivación por la derecha en reversa).
Mecanismo Típico	Funciones recursivas descendentes.	Autómata de pila (operaciones <i>Shift-Reduce</i>).
Complejidad de Implementación	Baja. Es muy intuitivo y fácil de programar manualmente sin herramientas.	Alta. Requiere construir complejas tablas de estado matemáticamente precomputadas.

Manejo de Recursividad	Prohíbe estrictamente la recursividad por la izquierda (entra en bucle infinito).	Soporta la recursividad por la izquierda de forma natural y eficiente.
Herramientas de Generación	ANTLR, JavaCC.	Bison, Yacc, CUP.

2.2. Evidencia de Uso Actual en la Industria

Ambas arquitecturas coexisten en el software moderno, dictadas por las necesidades específicas del lenguaje a compilar:

- **Evidencia de LL:** El lenguaje **Python** (a partir de la versión 3.9) migró su núcleo hacia un analizador sintáctico PEG (Parsing Expression Grammar), el cual es una variante del análisis descendente recursivo. Esta migración se justificó por la facilidad de mantenimiento frente a su antiguo parser LL(1). Por otro lado, la herramienta **ANTLR** (que genera parsers LL adaptativos) es el motor detrás de grandes corporaciones como Twitter para parsear millones de consultas de búsqueda estructuradas diarias.
- **Evidencia de LR:** **PostgreSQL** y **MySQL** utilizan parsers ascendentes (generados mediante Bison/Yacc) para interpretar el lenguaje SQL. Debido a que el estándar SQL posee una gramática extremadamente densa, con múltiples niveles de precedencia y ambigüedades estructurales, la robustez de las tablas LALR(1) es indispensable. El intérprete clásico de **Ruby** también mantiene su núcleo basado en un parser LR generado por Bison.

2.3. Definición del Lenguaje Sintético L y Ejemplos de Entrada

Para demostrar ambas estrategias operando sobre el mismo conjunto de reglas, definiremos un lenguaje "L" muy restrictivo que solo permite sumar dos variables y asignar el resultado.

Gramática del Lenguaje L:

- $S \rightarrow id = id + id ;$

Dos Ejemplos de Entrada Válidos:

1. $total = subtotal + iva ;$
2. $x = y + z ;$

2.4. Implementación del Lenguaje L (Python)

Para demostrar ambas estrategias operando sobre la gramática $S \rightarrow id = id + id ;$, procesaremos la entrada tokenizada ["id", "=", "id", "+", "id", ";"].

Implementación LL (Recursivo Descendente)

El analizador predice la regla y consume la cadena de tokens de izquierda a derecha.

Python

```
tokens = ["id", "=", "id", "+", "id", ";"]
```

```
posicion = 0
```

```
def match(tipo_esperado):
```

```
    """Verifica si el token actual coincide con el esperado y avanza el puntero."""
```

```
    global posicion
```

```
    if posicion < len(tokens) and tokens[posicion] == tipo_esperado:
```

```
        print(f"[LL] Match exitoso: {tokens[posicion]}")
```

```
        posicion += 1
```

```
        return True
```

```
    return False
```

```
def parse_asignacion():
```

```
    """Valida la estructura mediante llamadas recursivas simuladas."""
```

```
    print("Iniciando análisis LL (Top-Down)...")
```

```
    # Intenta validar la secuencia exacta exigida por la gramática
```

```
    if (match("id") and match("=") and match("id") and
```

```
match("+") and match("id") and match(";"):
```

```
print("-> Análisis LL completado con éxito. Sentencia válida.")
```

```
else:
```

```
print("-> Error de sintaxis detectado en el análisis LL.")
```

```
parse_asignacion()
```

Implementación LR (Ascendente Shift-Reduce)

El analizador utiliza una pila para ir desplazando (*Shift*) los tokens y colapsarlos (*Reduce*) de derecha a izquierda cuando coinciden con la regla.

```
Python
```

```
tokens = ["id", "=", "id", "+", "id", ";"]
```

```
pila = []
```

```
print("Iniciando análisis LR (Bottom-Up)...")
```

```
for token in tokens:
```

```
    # Acción SHIFT: empujar token a la pila
```

```
    pila.append(token)
```

```
    # Se imprime la pila invertida para emular la visualización LIFO (Tope a la izquierda)
```

```
    print(f"[LR] Acción SHIFT: {token:2} | Estado de pila: {pila[::-1]}")
```

```
    # Acción REDUCE: Verificar si el contenido de la pila coincide con la regla completa
```

```
    # Regla a reducir: id = id + id ;
```

```
    if len(pila) == 6:
```

```

if pila == ["id", "=", "id", "+", "id", ";"]:

    pila.clear()

    pila.append("S") # 'S' representa el símbolo no terminal inicial (Sentencia)

    print(f"[LR] Acción REDUCE: id = id + id ; -> S")

    print(f"[LR] Análisis LR completado con éxito. Pila final: [{pila[0]}]")

```

3. Resumen de Generadores de Analizadores Sintácticos y Gestión de Errores

Definición General Un metacompilador, o generador de analizadores sintácticos, es una herramienta de software que toma como entrada la descripción formal de un lenguaje (mediante una Gramática Libre de Contexto) y genera automáticamente el código ejecutable capaz de tokenizar y validar dicha estructura. Su propósito es abstraer la inmensa complejidad matemática de construir autómatas y tablas de estados manualmente. Ante las fallas de código introducidas por el humano, la responsabilidad principal de estas herramientas es la recuperación de errores, empleando clásicamente estrategias como el modo de pánico (ignorar tokens hasta un punto seguro) o la inserción heurística de tokens faltantes.

A continuación, se presentan cuatro generadores modernos e industriales (distintos a Flex, Bison, ANTLR, PLY o JavaCC) y sus sofisticados mecanismos de mitigación de errores:

Metacompilador (Python)	Arquitectura y Algoritmo	Gestión y Recuperación de Errores Sintácticos
Pegen	PEG (Parsing Expression Grammar). Es el generador oficial de CPython desde la versión 3.9.	Pegen abandona las técnicas adivinatorias. Gestiona los errores inyectando explícitamente "Reglas Inválidas" (<i>Invalid Rules</i>) dentro de la propia gramática. Si el código ingresa en un estado roto (ej: falta un paréntesis en un if), coincide matemáticamente con una regla de error preprogramada que detiene el parser y emite un mensaje altamente legible.

SLY (Sly Lex Yacc)	LALR(1) . Creado por David Beazley como el sucesor moderno y orientado a objetos de PLY.	Maneja los fallos mediante un método interceptor especial llamado <code>error(self, token)</code> . Cuando el analizador entra en pánico, delega el control a este método de clase, donde el programador puede inspeccionar el estado, descartar tokens problemáticos uno a uno o re-sincronizar el escaneo de entrada antes de obligar al parser a reanudar la ejecución.
Lark	Earley y LALR(1) . Es una de las bibliotecas de parsing más modernas y versátiles de Python.	Incorpora una potente interfaz interactiva denominada " <i>Puppet Parser</i> ". Ante un fallo sintáctico, Lark congela el autómata de pila y activa funciones <i>callback</i> definidas por el usuario (ej: <code>on_error</code>). Esto te permite inyectar tokens artificiales en caliente para "engañar" al parser, simulando que el usuario escribió correctamente el delimitador faltante y preservando el árbol AST intacto.
Arpeggio	PEG (Descendiente con Packrat Parsing). Basado puramente en objetos y expresiones de Python.	En lugar de entrar en pánico estructural, Arpeggio implementa un rastreador de fallos. Cuando el retroceso (<i>backtracking</i>) de PEG fracasa por completo, lanza una excepción de tipo <code>NoMatch</code> que contiene el historial exacto de la derivación. Proporciona con precisión milimétrica la fila, columna y contexto donde el árbol no pudo predecir el siguiente token.